

Python Dictionary – Complete Notes

◆ What is a Dictionary?

A dictionary is a collection used to store data in key–value pairs.

Key Points:

Data is stored as {key : value}

Keys must be unique and immutable (int, string, tuple)

Values can be mutable and duplicate

Dictionaries can store homogeneous and heterogeneous data

Defined using curly braces {}

Example:

```
d1 = {"name": "Raju", "age": 23}
```

```
print(type(d1))
```

Output:

```
<class 'dict'>
```

◆ Dictionary Rules

✗ Mutable objects (like list, set) cannot be keys

✓ Tuple can be used as a key (called a composite key)

```
d = {(1, 2): "apple"}
```

◆ Creating a Dictionary

1 Using Curly Braces

```
d = {"A": "apple", "B": "box"}
```

2 Using dict() Constructor

```
d = dict(A="apple", B="box")
```

◆ Nested Dictionary

A dictionary inside another dictionary.

```
d = {  
    "student1": {"name": "Ravi", "age": 22},  
    "student2": {"name": "Anil", "age": 23}  
}
```

◆ Accessing Dictionary Elements

1 Using Key

```
d = {"A": "apple", "B": "box"}  
print(d["A"])  
Output: apple
```

2 Using get()

Used to access values safely.

```
d = {"A": "apple", "B": ["v1", "v2", "v3"]}  
print(d.get("B")[1])  
Output: v2
```

If key not found:

```
d.get("C", "Invalid Key")
```

◆ Modifying a Dictionary

Change Existing Value

```
d = {"day1": "Sunday", "day2": "Monday"}  
d["day1"] = "Holiday"
```

Add New Key-Value Pair

```
d["day3"] = "Tuesday"
```

📌 Note:

If the key already exists, value will be updated, not added.

◆ update() Method

Used to add or update multiple key-value pairs.

```
d = {1: "one", 2: "two"}
```

```
d.update({3: "three"})
```

OR

```
new = {4: "four"}
```

```
d.update(new)
```

◆ Removing Elements from Dictionary

1 pop(key)

Removes a specific key and returns its value.

```
d = {1: "one", 2: "two"}
```

```
d.pop(1)
```

If key not found:

```
d.pop(5, "Invalid Key")
```

2 popitem()

Removes the last inserted key-value pair.

```
d.popitem()
```

✚ Raises error if dictionary is empty.

3 clear()

Removes all elements.

```
d.clear()
```

4 del Keyword

```
del d["key"] # deletes specific key
```

```
del d # deletes entire dictionary
```

◆ Copying Dictionary

`copy()` – Shallow Copy

```
d1 = {1: "one", 2: "two"}
```

```
d2 = d1.copy()
```

```
id(d1) != id(d2)
```

Changes in one do not affect the other

◆ Dictionary Access Methods

`keys()`

```
d.keys()
```

Returns all keys.

`values()`

```
d.values()
```

Returns all values.

`items()`

```
d.items()
```

Returns key-value pairs as tuples.

◆ Packing & Unpacking Dictionaries

Packing

```
d = {1: "one", 2: "two"}
```

Unpacking

```
print(*d)
```

Prints only keys.

◆ Merging Dictionaries

```
d1 = {1: "one"}
```

```
d2 = {2: "two"}
```

```
d3 = d1 | d2
```

✓ Summary

- ✓ Dictionaries store data as key-value pairs
- ✓ Keys must be unique and immutable
- ✓ Values can be mutable
- ✓ Supports nested structures
- ✓ Very useful for real-world data handling